Desenvolvimento de Sistemas Web II

Persistência, testes, OpenAPI, validação e arquitetura

DIM0547 — Turma 01 · Sprint 1 · parte 2 (vídeo)

Prof. Fernando · UFRN · 2026.2

Errata do material já publicado

Onde	O que corrigir
Slides 04, "Bloqueante vs Não bloqueante"	No Quarkus, método que devolve tipo comum (`List<Tarefa>`, `Response`) roda numa **thread de trabalho**, não no event loop. Detalhe no próximo slide
Slides 04, "Para a próxima aula"	O OpenAPI do Ktor, no exemplo e no MUSI, é o gerador **nativo** (`describe`). O `ktor-openapi` (smiley4) é a alternativa da comunidade
Slides 04, "Passo 1"	O exemplo responde `no ar` em `GET /`; o slide mostrou `GET /tarefas`
Slides 04, "Passo 3"	O `POST` usa a lista direto; `repositorio.adicionar` só aparece no Passo 4
Cronograma	Entrega da Sprint 1 adiada para **16/10 (sexta), 23:59**. A aula de 23/09 foi cancelada; em 28 e 30/09, no lugar das apresentações, houve uma *daily meeting* online com cada grupo. O semestre passa a ter só mais uma sprint, a de novembro

Errata: qual thread executa o método no Quarkus

Quem decide é a **assinatura** do método (conferido pelo nome da thread):

O método devolve	Roda em	Nome da thread
objeto comum: `List<Tarefa>`, `String`, `Response`	thread de trabalho (pool grande)	`executor-thread-1`
`Uni`, `Multi`, `CompletionStage`	event loop (poucas threads)	`vert.x-eventloop-thread-1`

- `@Blocking` e `@NonBlocking` trocam a escolha; `@Transactional` conta como bloqueante
- O exemplo, imperativo, roda no modelo de **uma thread por requisição**, com um pool grande
- **Bloquear dentro de um método reativo** prende o event loop

📖 **Ref.** `leituras/web2-s1-pte1.md`, seção 9.4 · Quarkus — REST: execution model

Onde paramos

No vídeo da parte 1, a API de tarefas nos dois stacks, **passos 1 a 4**:

```
Rotas (DSL) × recursos (anotações)
JSON, POST, 201 com o corpo criado
RepositorioDeTarefas: a porta
RepositorioEmMemoria: a implementação
```

Hoje	O que entra
5	Injeção de dependência: Koin × CDI
6 e 7	PostgreSQL, Flyway, Exposed × Panache
8	Testes: sem banco e com banco de verdade
9 e 10	OpenAPI · o ambiente em tasks do `mise`
11 e 12	Validação em *problem details* · camadas e teste de arquitetura

Passo 5

Injeção de dependência

Passo 5 — Koin × CDI

Ktor + Koin — grafo em código

```
fun Application.configurar(
    repositorio: RepositorioDeTarefas,
) {
  install(Koin) {
    modules(module {
      single<RepositorioDeTarefas> { repositorio }
    })
  }
  install(ContentNegotiation) { json() }
  rotas()
}

// Rotas.kt
val repositorio by inject<RepositorioDeTarefas>()
```

Quarkus + CDI — por anotação

```
@ApplicationScoped
public class RepositorioEmMemoria
    implements RepositorioDeTarefas { }

// RecursoDeTarefas.java
@Inject
RepositorioDeTarefas repositorio;
```

Falta quem forneça `RepositorioDeTarefas`	Koin	CDI
O erro aparece	ao **subir** a aplicação	no **build** (`Unsatisfied dependency`)

`configurar(repositorio)` recebe a porta de fora: é o que vai deixar o teste de rota rodar **sem banco** (Passo 8).

O que a lista em memória não resolve

Problema	Com PostgreSQL
Dados somem ao reiniciar	Tarefa criada continua lá após reiniciar
Cada instância vê os seus dados	Todas leem o mesmo banco
Concorrência	2.000 `POST` simultâneos: **2.000 × `201`, ids únicos**

> Em 14/09, os mesmos 2.000 `POST` perderam tarefas e repetiram ids. Quem garante agora é o banco: `SERIAL` e `INSERT` atômico.

A porta, os adaptadores



- Uma classe nova implementa a mesma interface do Passo 4
- A implementação em memória **fica**: é a dos testes sem Docker

Passo 6

Banco: JDBC, pool e onde fica a senha

JDBC e pool de conexões

```
jdbc:postgresql://localhost:5432/tarefas
     └── driver ─┘ └─ host ─┘ └porta┘ └ banco ┘
```

Ktor — o pool é escolha sua

```
HikariDataSource(HikariConfig().apply {
    jdbcUrl = config.url
    username = config.usuario
    password = config.senha
    maximumPoolSize = 5
})
```

Quarkus — vem com a extensão

```
quarkus.datasource.db-kind=postgresql
%prod.quarkus.datasource.jdbc.url=\
  ${DB_URL:jdbc:postgresql://...}
```

Em dev e test, **sem URL**: Dev Services

URL, usuário e senha em **variável de ambiente**. Nunca no repositório.

Passo 6

Migrações: o esquema sob controle de versão

Flyway

```
src/main/resources/db/migration/
  V1__cria_tarefas.sql
  V2__adiciona_prazo.sql
```

- Aplica em ordem; guarda versão e **checksum** em `flyway_schema_history`
- Primeira subida: `Migrating schema "public" to version "1 - cria tarefas"`
- Seguintes: `Schema "public" is up to date. No migration necessary.`

Ktor	Quarkus
`Flyway.configure().dataSource(ds).load().migrate()`	`quarkus.flyway.migrate-at-start=true`
+ `flyway-database-postgresql`	já incluso na extensão

O esquema é da migração, não do ORM

A rubrica: **esquema só por migração**, sem `ddl-auto` ou equivalente.

- Quarkus: `quarkus.hibernate-orm.schema-management.strategy=none`
- Ktor: o `object Tarefas` descreve a tabela para as consultas; **nunca** `SchemaUtils.create`

Nunca edite uma migração aplicada. O Flyway recusa subir:

```
Migration checksum mismatch for migration version 1
-> Applied to database : 1556295178
-> Resolved locally    : 1950589099
```

> Mudou o esquema? `V2__...`. No PostgreSQL, uma migração que falha é **desfeita inteira**.

Passo 7

Exposed × Panache: o mesmo adaptador, dois estilos

Ktor: Exposed (DSL)

```
object Tarefas : Table("tarefas") {
    val id = integer("id").autoIncrement()
    val titulo = varchar("titulo", 200)
    val feita = bool("feita").default(false)
    override val primaryKey = PrimaryKey(id)
}

override suspend fun buscar(id: Int): Tarefa? = suspendTransaction(db) {
    Tarefas.selectAll().where { Tarefas.id eq id }.singleOrNull()?.paraTarefa()
}
```

- Toda operação dentro de uma **transação**; fora dela: `No transaction in context.`
- A porta ganhou **`suspend`**: JDBC bloqueia, `suspendTransaction` não trava a thread do servidor
- Exposed 1.x: pacotes `org.jetbrains.exposed.v1...`

Quarkus: Hibernate com Panache

```java
@Entity @Table(name = "tarefas")
public class TarefaEntidade extends PanacheEntityBase {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) public Integer id;
    public String titulo;
    public boolean feita;
}

@ApplicationScoped
public class RepositorioPanache
        implements RepositorioDeTarefas, PanacheRepositoryBase<TarefaEntidade, Integer> {
    @Transactional
    public Tarefa adicionar(NovaTarefa nova) { /* new TarefaEntidade(); persist(e); */ }
}
```

- Escrita **sem** `@Transactional`: `TransactionRequiredException` → `500`
- A entidade não é o `record Tarefa`: JPA exige construtor vazio e campos mutáveis

Lado a lado

	Ktor + Exposed	Quarkus + Panache
Estilo	SQL em Kotlin	ORM: objeto ↔ tabela
Transação	`suspendTransaction { }` explícito	`@Transactional`
Pool	HikariCP, configurado por você	Agroal, pela extensão
Banco em dev	`docker compose up -d`	Dev Services, automático
Banco nos testes	Testcontainers	Dev Services
Esquema	nunca `SchemaUtils.create`	`strategy=none`

> Exposed: cada SQL visível. Panache: CRUD pronto, com atenção ao que o Hibernate faz por baixo (N+1, carregamento preguiçoso).

O banco como última barreira

Em 14/09, o Quarkus aceitava `POST {}` e criava tarefa com título `null`.

Agora a coluna é `NOT NULL`:

```
ERROR: null value in column "titulo" of relation "tarefas"
       violates not-null constraint
```

Pedido	Ktor	Quarkus
`POST {}`	`400` (kotlinx exige `titulo`)	**`500`**

> O dado ruim não entrou, mas o cliente recebeu erro **do servidor**. A primeira barreira é **validar** — critério da rubrica, e o Passo 11.

Passo 8

Testes: da rota ao banco de verdade

Duas camadas de teste

Rota sem banco `ms`

```
testApplication {
  application {
    configurar(RepositorioEmMemoria())
  }
  client.get("/tarefas/abc") // 400
}
```

Quarkus: JUnit puro, `new RecursoDeTarefas()` + repositório em memória

Integração `banco real`

```
val postgres = PostgreSQLContainer(
    "postgres:17-alpine").apply { start() }

modulo(ConfigBanco(postgres.jdbcUrl,
  postgres.username, postgres.password))
```

Quarkus: `@QuarkusTest` + **Dev Services**, nenhuma linha sobre banco

> Rubrica: unitários + Testcontainers, **passando no Docker Desktop e no CI** sem mudar configuração.

Testcontainers na prática

- Container por **classe** (no `companion object`): 4 testes em 2,6 s com a imagem baixada
- Primeira vez baixa a imagem (a `postgres:17` tem 158 MB)
- Testcontainers 2.x: `testcontainers-postgresql`, pacote `org.testcontainers.postgresql`

Cuidado com o Docker Engine 29 (Docker Desktop atualizado):

```
Could not find a valid Docker environment
UnixSocketClientProviderStrategy: failed with exception BadRequestException (Status 400)
```

O Quarkus 3.28 traz o Testcontainers **1.21.3**, que não conversa com o Docker Engine 29. Correção: importar o BOM do Testcontainers **1.21.4** antes do BOM do Quarkus.

Passo 9

OpenAPI: a documentação que sai do código

Ktor × Quarkus

Ktor — `describe` (experimental no 3.5)

```
get("/{id}") { /* ... */ }.describe {
  summary = "Busca uma tarefa pelo id"
  parameters {
    path("id") { schema = jsonSchema<Int>() }
  }
}
swaggerUI(path = "docs") { /* ... */ }
```

`/docs` · `/docs/documentation.yaml`

Quarkus — só a extensão

```
@GET @Path("/{id}")
@Operation(summary = "Busca uma tarefa pelo id")
public Tarefa buscar(@PathParam("id") int id)
```

`/q/openapi` · `/q/swagger-ui` (dev/test; em prod, `always-include=true`)

> Sem declarar o tipo, o `{id}` do Ktor sai como `string`.

Rodar tudo junto

```
docker compose up -d          # PostgreSQL 17, volume "dados"
./gradlew run                 # Ktor, :8080
mvn quarkus:dev               # Quarkus, :8081 (Dev Services)
```

```
curl -s -i -X POST localhost:8080/tarefas \
  -H 'Content-Type: application/json' -d '{"titulo":"Persistir"}'
```

Os exemplos: `exemplos/ktor-tarefas` e `exemplos/quarkus-tarefas`, **passos 5 a 12** do `PASSOS.md`.

> **Sem Java 25 ou Docker na máquina, ou no laboratório:** crie um Codespace do repositório (Code → Codespaces). Ele traz Java, Maven, Go e Docker, com tudo já baixado; os comandos são os mesmos, e as portas 8080 e 8081 aparecem na aba **Portas**. Pare o Codespace ao terminar: o uso gratuito mensal é limitado.

Passo 10 — o ambiente em tasks

```
# mise.toml, na raiz do repositório
[tools]
java  = "temurin-25"
maven = "3.9"

[tasks."ktor:banco"]
dir = "exemplos/ktor-tarefas"
run = "docker compose up -d"
```

```
mise install          # a mesma versão do JDK e do Maven para todo mundo
mise tasks            # a lista: ktor:banco, ktor:run, ktor:test, quarkus:dev, quarkus:test...
mise run test         # os testes dos dois exemplos, como no CI
```

Nada muda no código: cada task é o comando dos passos anteriores, com nome. O ganho é o `[tools]` — e é a mesma tarefa T3 da Sprint 1 (`mise run up`).

📖 **Ref.** mise — tasks

Passo 11

Validação e erros em *problem details*

400 ou 422?

O que está errado	Status	Exemplo
A **forma**: JSON quebrado, campo faltando, tipo errado	`400`	`{}` no Ktor · `{"titulo":[1]}`
A **regra**: a forma está certa, o conteúdo não	`422`	`{"titulo":"  "}`
O recurso não existe	`404`	`GET /tarefas/9999`
Erro nosso	`500`, **sem** detalhe interno	exceção não prevista

Todos no formato da **RFC 9457**, `application/problem+json`:

```
{"type":"/problemas/entrada-invalida","title":"Entrada inválida","status":422,
 "detail":"A entrada viola 1 regra(s).","violacoes":["titulo: não pode ficar em branco"]}
```

> `type` identifica a **classe** do erro; `detail`, esta ocorrência. `violacoes` é um membro de extensão, que a RFC permite.

📖 **Ref.** RFC 9457 — Problem Details for HTTP APIs

A regra no domínio, a resposta num lugar só

Ktor · `StatusPages`

```kotlin
// Tarefa.kt: Kotlin puro
fun violacoes(): List<String> = when {
  titulo.isBlank() ->
    listOf("titulo: não pode ficar em branco")
  titulo.length > 200 ->
    listOf("titulo: no máximo 200 caracteres")
  else -> emptyList()
}

// Rotas.kt: a rota só lança
if (violacoes.isNotEmpty())
  throw EntradaInvalida(violacoes)

// Erros.kt
install(StatusPages) {
  exception<EntradaInvalida> { call, e ->
    call.responderProblema(
      HttpStatusCode.UnprocessableEntity, ...)
  }
}
```

Quarkus · `@ServerExceptionMapper`

```java
// NovaTarefa.java: Java puro
public List<String> violacoes() {
  if (titulo == null || titulo.isBlank())
    return List.of(
      "titulo: não pode ficar em branco");
  ...
}

// RecursoDeTarefas.java: o recurso só lança
if (!violacoes.isEmpty())
  throw new EntradaInvalida(violacoes);

// Erros.java
@ServerExceptionMapper
public Response entradaInvalida(
    EntradaInvalida e) {
  return problema(422, ...);
}
```

As mesmas perguntas, antes e depois

Pedido	Ktor antes → depois	Quarkus antes → depois
`POST {"titulo":"  "}`	`201` → **`422`**	`201` → **`422`**
`POST {}`	`400` → `400` *problem*	**`500`** → **`422`**
`GET /tarefas/abc`	`400` → `400` *problem*	`404` → `404` *problem*
`GET /tarefas/9999`	`404` → `404` *problem*	`404` → `404` *problem*
`POST` válido	`201` → `201` + `Location`	`201` → `201` + `Location`

- O `500` do Quarkus some: a entrada inválida **não chega** ao banco
- `Location: /tarefas/{id}` no `201`, como pede a tarefa T5
- Alternativa no Quarkus: **Bean Validation** (`@NotBlank`, `@Valid`); o MUSI usa as duas

Passo 12

Camadas em pacotes e o teste de arquitetura

A regra de dependência, em pacotes

```
br.ufrn.exemplo.tarefas
├── dominio/          Tarefa, NovaTarefa, EntradaInvalida, RepositorioDeTarefas
└── adaptadores/
    ├── http/         rotas ou recurso, erros
    ├── banco/        Exposed ou Panache
    └── memoria/      RepositorioEmMemoria
```

```
  adaptadores  ──────►  dominio        a seta só aponta para dentro
```

- O **domínio** não importa framework web, banco nem injeção de dependência
- No Ktor, `Aplicacao.kt` (na raiz) liga as camadas; no Quarkus, o CDI
- Pacote é só convenção: sem um teste, nada impede um `import` errado

ArchUnit: a regra vira teste

Ktor

```
noClasses().that()
  .resideInAPackage("..dominio..")
  .should().dependOnClassesThat()
  .resideInAnyPackage(
    "io.ktor..", "org.koin..",
    "org.jetbrains.exposed..",
    "java.sql..", "..adaptadores..",
  )
```

Quarkus

```
noClasses().that()
  .resideInAPackage("..dominio..")
  .should().dependOnClassesThat()
  .resideInAnyPackage(
    "jakarta..", "io.quarkus..",
    "org.hibernate..", "java.sql..",
    "..adaptadores..");
```

Um `import jakarta.ws.rs.core.Response` no domínio, e `mvn test`:

```
Architecture Violation [Priority: MEDIUM] – Rule 'no classes that reside in a package
'..dominio..' should depend on classes that reside in any package ['jakarta..', ...]'
was violated (3 times)
```

A rubrica pede que o teste **falhe** quando a regra é violada: o exemplo prova isso com uma classe-fixture só dos testes (`violacao/dominio/Contaminado`).

📖 **Ref.** ArchUnit — User Guide

No MUSI: o que o exemplo não tem

O que a rubrica pede	Onde ver
Duas entidades com relacionamento	`Obra` 1:N `Anotação`, com FK e `ON DELETE CASCADE`
Rota aninhada	`/obras/{id}/anotacoes/{anotacaoId}`
Paginação e filtros no SQL	`?pagina=&tamanho=&ordem=&artista=&dimensao=&valor=`
A mesma migração em dois stacks	`db/migration` igual nos dois; o CI compara com `diff -r`
Teste de arquitetura	ArchUnit no Ktor **e** no Quarkus
Decisão registrada	`docs/decisoes/0004-persistencia-postgresql-flyway.md`

`github.com/fmarquesfilho/musi` · a busca por faceta continua no serviço Go, em `/busca`

> Sem banco configurado, as duas APIs sobem assim mesmo: a busca funciona e o CRUD responde `503`.

Entrega da Sprint 1 — 16/10, 23:59

Critério	Peso
CRUD de ≥ 2 entidades com relacionamento, paginação e filtros	25%
Camadas + teste de arquitetura no CI	25%
Flyway, banco via `docker compose`, esquema só por migração	20%
Unitários + Testcontainers, local **e** no CI	20%
Validação, *problem details*, OpenAPI	10%

Guia e tarefas: `docs/SPRINT-1.md` e `docs/SPRINT-1-TAREFAS.md`. Exemplos: passos 11 e 12 para validação e arquitetura; MUSI para duas entidades, paginação e filtros.

O que muda no semestre

	Antes	Agora
Entrega da Sprint 1	02/10	**16/10** (sexta), 23:59
Depois da Sprint 1	Sprint 2, Sprint 3 e bloco final	**só a Sprint 2**, que é a entrega final, em **30/11**
Prova escrita	21/10	**09/11** (segunda), em laboratório
Prova de reposição	30/11	**02/12** (quarta)
Fim de cada sprint	apresentação por coorte	***daily meeting*** online com cada grupo, como em 28 e 30/09
Unidades	três sprints e duas provas espalhadas	U1 = Sprint 0 (30%) + Sprint 1 (70%) · U2 = prova · U3 = Sprint 2

- Dentro de cada sprint, **nada muda**: entrega técnica 50%, atividade no repositório 30%, comunicação 20%
- A *daily meeting* entra onde antes entrava a apresentação; as de 28 e 30/09 valeram para a Sprint 1
- Vale a **maior nota** entre a prova e a reposição

> Tudo está em `docs/CRONOGRAMA.md`, `docs/AVALIACAO.md` e `docs/RUBRICAS.md`.

O calendário até dezembro

Semana	Segunda	Quarta
05 e 07/10	🟢 em sala: conteúdo da Sprint 1	🟢 em sala: conteúdo da Sprint 1
12 e 14/10	feriado	🔵 online: acompanhamento · 🚀 **sexta, 16/10: entrega da Sprint 1**
19 e 21/10	🟢 em sala: conteúdo da Sprint 2	🟢 em sala: conteúdo da Sprint 2
26 e 28/10	🔵 online: acompanhamento	feriado
02 e 04/11	feriado	🔵 online: revisão para a prova
09 e 11/11	🟢 em sala: **prova escrita**	🔵 online: acompanhamento
16 e 18/11	🔵 online: acompanhamento	🟢 em sala: oficina de projeto
23 e 25/11	🔵 online: *daily meetings*	🔵 online: *daily meetings*
30/11 e 02/12	a definir · 🚀 **entrega final**	🟢 em sala: **prova de reposição**

A Sprint 2, a última

Critério	Peso
Microsserviço Go, com responsabilidade justificada	25%
Contrato Protobuf e integração gRPC, com teste automatizado	30%
Sistema no ar: API, serviço Go e banco gerenciado, com URL pública	25%
Ambiente e testes: `docker compose up` e suíte verde local e no CI	10%
Prontidão do repositório: README, OpenAPI, 2 ADRs novas	10%

> O que vocês entregarem em 30/11 é o produto final. A prova de 09/11 cobre as Sprints 0, 1 e 2.

Onde estudar depois

Fonte	Foco
`leituras/web2-s1-pte2.md`	Esta aula, com erros comuns e exercícios
`jetbrains.com/help/exposed`	Exposed: DSL, transações
`quarkus.io/guides/hibernate-orm-panache`	Panache
`documentation.red-gate.com/flyway`	Flyway
`java.testcontainers.org` · `quarkus.io/guides/databases-dev-services`	Testcontainers e Dev Services
`ktor.io/docs/openapi-spec-generation.html` · `quarkus.io/guides/openapi-swaggerui`	OpenAPI